Cipher Rekeying via Closures

Alexander Towell
lex@metafunctor.com

June 9, 2026

Abstract

A cipher value is opaque to anyone without the trapdoor secret that encoded it. Existing trapdoor-computing frameworks treat the secret as a fixed parameter: one secret per system, established at setup, never changed. Real systems rotate keys, share data across trust boundaries, and retire old secrets without going through plaintext. We develop *cipher rekeying*: a transformation that maps a cipher value encoded under one secret to a cipher value encoded under another, preserving the latent value. We frame the problem through the closures-as-data-structures lens of Abelson and Sussman [1]: cipher values, cipher maps, and cipher data structures are all closures parameterized by a secret, and rekeying is a natural transformation between such closures. The construction is straightforward when the trusted machine knows both secrets: the rekeying functor is itself a cipher map, and once constructed it can be applied by the untrusted machine to any cipher value. The information-theoretic cost is precise: rekeying introduces correlation leakage across cipher spaces, and the orbit-closure bound of [11] extends directly to bound the residual confidentiality. We prove that rekeying commutes with arbitrary cipher operations (naturality), characterize the leakage of an n-step rekeying chain, and show how multiple representations recover confidentiality at the cost of representation multiplicity. The contribution is the rekeying primitive plus its information-theoretic accounting, neither of which appear in the existing trapdoor-computing literature.

1 Introduction

In Abelson and Sussman's classic exposition [1], the pair (x, y) is implementable as a closure:

$$\mathrm{cons}(x, y) = \lambda m.\, m(x, y), \qquad \mathrm{car}(p) = p(\lambda x\, y.x), \qquad \mathrm{cdr}(p) = p(\lambda x\, y.y).$$

The data structure is a function that captures its components and exposes operations. This duality, between data structures and functions that implement them, is the conceptual move that lets us speak about cipher data structures without first having to build a separate theory of cipher storage.

Trapdoor computing [12] provides the cipher machinery: a cipher value $c \in \{0,1\}^n$ is an opaque bit string that encodes some latent value x under a secret s, decodable only by a holder of s. A cipher map $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is an opaque function that realizes a latent function $f : X \to Y$ over cipher values: it can be evaluated by an untrusted machine without revealing x, f, or s. Algebraic cipher types [11] extend this to products, sums, and exponentials, with confidentiality bounds via orbit closure.

What's missing, in the existing trapdoor-computing literature, is the secret as a first-class object. The current papers treat the secret as a fixed parameter: encoded into $\mathrm{enc}, \mathrm{dec}, \hat{f}$ at construction time and never thought about again. Real systems do not work this way. They rotate keys on a schedule. They share cipher values across organizational boundaries where the two parties hold

different secrets. They retire old keys without going through plaintext. None of these operations is supported by the current formalism.

This paper develops the missing piece: *cipher rekeying*, the operation that takes a cipher value $c \in \mathsf{C}_{s_1}(X)$ encoded under one secret s_1 and produces a cipher value $c' \in \mathsf{C}_{s_2}(X)$ encoded under a different secret s_2, such that c and c' encode the same latent value x. We frame rekeying through the SICP closure abstraction, treating cipher values, cipher maps, and cipher data structures uniformly as closures over a secret. Rekeying is a natural transformation between such closures: it commutes with the operations the closures expose.

Contributions.

1. **Cipher closures** (Section 2). We use the cipher-closure view: cipher values, cipher maps, and cipher data structures are all procedures that capture a secret, so the secret is captured state that rekeying re-parameterizes. The general code–data duality of cipher closures (the dispatch pattern, cipher data structures, designed orbits) is developed separately [13].

2. **The rekeying functor** (Section 3). We define the rekeying functor $r_{s_1 \to s_2}$ as a transformation between cipher closures with different secrets. The functor is sound when it preserves the underlying latent values through both decoders.

3. **Cipher-map-mediated rekeying** (Section 4). We give a concrete construction: $r_{s_1 \to s_2}$ is itself a cipher map, constructed by the trusted machine from both secrets, and applied by the untrusted machine to any cipher value needing rekeying. Construction cost is one cipher map; per-rekeying cost is one cipher map evaluation.

4. **Naturality** (Section 5). We prove that rekeying commutes with arbitrary cipher operations (Theorem 5.1): for any latent function f, the diagram $r_{s_1 \to s_2} \circ \mathsf{C}_{s_1}(f) = \mathsf{C}_{s_2}(f) \circ r_{s_1 \to s_2}$ holds, where $\mathsf{C}_{s_i}(f)$ is the cipher map realizing f under secret s_i. This means rekeying can be performed before, after, or between cipher operations without affecting the result.

5. **Information-theoretic cost** (Section 6). We characterize the leakage rekeying introduces: the cross-cipher-space correlation between c and $r_{s_1 \to s_2}(c)$ is observable to the untrusted machine and reduces confidentiality. We extend the orbit-closure bound of [11, Thm. 5.3] to rekeying chains, and show that multiple representations smear the correlation at the cost of representation multiplicity.

6. **Applications** (Section 7). We sketch four applications: scheduled key rotation, multi-tenant cipher systems, forward secrecy via key retirement, and rekeying chains for evolving trust boundaries.

What this is not. This paper does not develop a public-key proxy re-encryption scheme [2]. The rekeying construction here requires the trusted machine to know both secrets at the moment of construction: it is not a third-party operation. The advantages over public-key PRE are different: information-theoretic guarantees instead of computational ones, no algebraic key structure required, and a clean composition with the rest of the trapdoor-computing framework. Section 8 addresses the comparison in detail.

2 Preliminaries

We recall the cipher map abstraction from [12] and the algebraic-types extension from [11], and fix the closure notation we use throughout.

2.1 Cipher maps

Definition 2.1 (Cipher map [12, Def. 3.1])**.** A *cipher map* for a latent function $f : X \to Y$ is a tuple $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ where $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is a total function on n-bit strings, $\mathrm{enc} : X \times \{0, \ldots, K(x){-}1\} \to \{0,1\}^n$ is an encoding function, $\mathrm{dec} : \{0,1\}^n \to Y \cup \{\perp\}$ is a decoding function, and s is the trapdoor secret. The untrusted machine holds $\hat{f}$. The trusted machine holds enc, dec, and s.

A cipher map satisfies four properties parameterized by $(\eta, \varepsilon, \mu, \delta)$ [12, Sec. 4]: totality, representation uniformity (δ-bounded), correctness (η-bounded), and composability. We write $\mathsf{C}_s(\cdot)$ for the *cipher functor* at secret s, dropping the subscript to $\mathsf{C}(\cdot)$ when s is clear from context. On a latent type X it returns the cipher space $\mathsf{C}_s(X)$, the set of cipher values that encode elements of X under s. On a latent function $f : X \to Y$ it returns the cipher map realizing f, which we also write $\hat{f}$; the two names denote the same arrow, $\mathsf{C}_s(f) = \hat{f}$. The two faces cohere, because the cipher exponential $\mathsf{C}_s(X \to Y)$ is exactly the space of cipher maps $\mathsf{C}_s(X) \to \mathsf{C}_s(Y)$ [11, Prop. 4.3]: the bit-string function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is at once the functor's value $\mathsf{C}_s(f)$ and a point of the cipher type $\mathsf{C}_s(X \to Y)$. We default to $\hat{f}$ for cipher maps and $\mathsf{C}_s(X)$ for cipher spaces, reserving the functor form $\mathsf{C}_s(f)$ for when its action on morphisms is the point. When two distinct secrets s_1, s_2 are in play, we abbreviate the encoder and decoder at s_i to enc_i and dec_i.

2.2 Closures over a secret

A closure, in the SICP sense [1], is a function that has captured (closed over) values from its lexical environment. Throughout this paper we work with closures that capture a secret s and expose an operation interface.

Definition 2.2 (Cipher closure)**.** A *cipher closure* over secret s is a function

$$\kappa_s : \mathcal{O} \to \mathcal{R}$$

where $\mathcal{O}$ is a set of operations and $\mathcal{R}$ is the union of result types associated with each operation. The closure has access to s and any auxiliary state captured at construction time. The signature of κ_s is determined by the operations it supports; we write $\kappa_s : \mathcal{O} \to \mathcal{R}$ abstractly, with the understanding that each operation $o \in \mathcal{O}$ has its own input and output types.

Example 2.1 (Cipher Boolean as closure). The cipher Boolean type $\mathsf{C}(\mathrm{Bool})$ of [11, Sec. 7.1] is a partition of $\{0,1\}^n$ into True, False, and Noise regions. A cipher Boolean value b is realized as a closure that captures the secret s (which determines the partition) and the bit string itself, exposing operations such as

- `decode`(): maps the captured cipher value to its underlying Boolean. This operation is invocable only by a party that holds the secret s (the trusted machine), since it requires the partition; the untrusted machine, which does not hold s, simply does not have access to a working `decode` at all.
- `and`$(b') \to \mathsf{C}(\mathrm{Bool})$: cipher Boolean AND with another cipher Boolean, realized by a cipher map.

- $\texttt{not}() \to \mathsf{C}(\text{Bool})$: cipher Boolean NOT, realized by a cipher map.

The `and` and `not` operations are exposed to the untrusted machine and inherit the orbit-closure leakage of cipher Boolean operations [11, Ex. 5.1]: an adversary that computes $\texttt{and}(b, \texttt{not}(b))$ obtains a cipher encoding of False, usable to partition other cipher Booleans by value.

Example 2.2 (Cipher pair via SICP construction). Following Abelson and Sussman, a cipher pair of cipher values $a \in \mathsf{C}(A)$ and $b \in \mathsf{C}(B)$ can be implemented as the closure

$$\text{cipher\_cons}(a, b) \;=\; \lambda m.\, m(a, b).$$

The projections are

$$\text{cipher\_first}(p) \;=\; p(\lambda a\, b.\, a), \qquad \text{cipher\_second}(p) \;=\; p(\lambda a\, b.\, b).$$

This realizes the componentwise product $\mathsf{C}(A) \times \mathsf{C}(B)$ of [11, Sec. 4.2]: projections are *operationally* free (no cipher map is invoked to extract a or b, the closure simply returns its captured component). They are not *leakage-free*: the adversary now has both cipher values from one closure, and the joint distribution of cipher pairs across many closures converges, in the large-sample limit, to the joint distribution of latent values (a, b) [11, Prop. 4.1]. The joint product $\mathsf{C}(A \times B)$ instead encodes the pair as a single cipher value, hiding the correlation at the cost of cipher-map-mediated projections. This is the encoding-granularity trade-off of [12, Sec. 8.1], made visible in closure form: componentwise captures two cipher values, joint captures one plus the projection maps.

Example 2.3 (Mutable cipher counter). A closure can capture mutable state. A cipher counter starting at 0 is the closure

$$\text{cipher\_counter}() \;=\; \texttt{let}\ c = \text{enc}(0, 0)\ \texttt{in}\ \lambda\texttt{op}.\ \begin{cases} c & \texttt{op} = \texttt{read} \\ c \leftarrow \widehat{\text{succ}}(c);\ c & \texttt{op} = \texttt{increment} \end{cases}$$

where $\widehat{\text{succ}}$ is the cipher successor map. The closure mutates the captured cipher value c in place. This shows that closures are not restricted to pure functions: with captured mutable state, they realize mutable cipher data structures.

Remark 2.1 (The counter leaks its orbit). Example 2.3 illustrates closure-based mutability, not confidentiality. An adversary with active access to the closure's operations and a known initial state (zero is plaintext) can call `increment` repeatedly from the known starting cipher value, recovering the orbit $\{c_0, \widehat{\text{succ}}(c_0), \widehat{\text{succ}}^2(c_0), \ldots\}$, and pair each cipher value with its plaintext index. This is exactly the orbit-closure leakage of [11, Ex. 5.2] (successor on a bounded type). The closure abstraction does not weaken or strengthen this leakage; it packages the operations through which the orbit is reached, and [11, Thm. 5.3] bounds the resulting confidentiality. Mitigations are the standard ones for orbit-closure leakage: randomize the initial cipher value (so the orbit's starting point is not pinned to plaintext zero), use multiple representations per plaintext (so $\widehat{\text{succ}}$ is randomized and the orbit fans out instead of forming a single chain), or cap the orbit through typed composition chains ([11, Sec. 5.5], distinct cipher spaces per counter level). None of these affect mutability; they reduce per-operation leakage.

These examples illustrate that cipher values, cipher maps, and cipher data structures are all cipher closures over a secret. The general theory — the dispatch pattern by which a multi-operation data structure becomes a single cipher map, designed orbits, and the code–data duality — is developed separately [13]. For rekeying we need only the simplest reading: a cipher object is parameterized by the secret it captured at construction, and rekeying re-parameterizes that secret.

3 The Rekeying Functor

We now define cipher rekeying. The setup: two secrets s_1, s_2, two cipher spaces $\mathsf{C}_{s_1}(X)$ and $\mathsf{C}_{s_2}(X)$, and the question of how to move between them.

3.1 Definition

Definition 3.1 (Rekeying functor)**.** Let X be a latent value type, s_1 and s_2 two distinct secrets, and $\mathsf{C}_{s_1}(X)$, $\mathsf{C}_{s_2}(X)$ the corresponding cipher spaces. A *rekeying functor* from s_1 to s_2 is a function

$$r_{s_1 \to s_2} : \mathsf{C}_{s_1}(X) \to \mathsf{C}_{s_2}(X)$$

such that for every $c \in \mathsf{C}_{s_1}(X)$,

$$\mathrm{dec}_1(c) \;=\; \mathrm{dec}_2\big(r_{s_1 \to s_2}(c)\big).$$

That is, $r_{s_1 \to s_2}$ takes any cipher value encoded under s_1 to a cipher value encoded under s_2 that decodes to the same latent value.

We call $r_{s_1 \to s_2}$ a *functor* because it sends each cipher space $\mathsf{C}_{s_1}(X)$ to $\mathsf{C}_{s_2}(X)$ and commutes with cipher operations, the natural-transformation condition Theorem 5.1 establishes [8]. The categorical reading organizes the discussion but is not needed for the results.

3.2 Three regimes for rekeying

The definition is implementable in three distinct regimes, distinguished by who knows what at construction time.

Regime 1: trusted-machine rekeying. $r_{s_1 \to s_2}$ is computed by the trusted machine, which holds both secrets. For each c, the trusted machine computes $\mathrm{dec}_1(c)$, then $\mathrm{enc}_2(\cdot)$ on the result. This works in all cases but is expensive: every cipher value passes through plaintext on the trusted machine.

Regime 2: cipher-map-mediated rekeying. The trusted machine constructs a cipher map $\hat{r}$ that realizes $r_{s_1 \to s_2}$. Once constructed, $\hat{r}$ is handed to the untrusted machine, which can apply it to any cipher value in $\mathsf{C}_{s_1}(X)$ without knowing either secret. Construction cost is one cipher map (over the domain X); per-rekeying cost is one cipher map evaluation.

Regime 3: third-party rekeying. $\hat{r}$ is constructable by a third party who holds neither secret. This is proxy re-encryption [2] in the classical literature. Information-theoretically it is impossible: if a third party can produce a function from $\mathsf{C}_{s_1}(X)$ to $\mathsf{C}_{s_2}(X)$ that preserves latent values, the third party effectively has a decoder for s_1 and an encoder for s_2. Public-key constructions sidestep this with computational hardness assumptions, which we do not adopt.

This paper develops Regime 2 in full. Regimes 1 and 3 appear only in the related-work comparison (Section 8).

4 Cipher-Map-Mediated Rekeying

We give the construction for Regime 2 and analyze its cost.

4.1 Construction

Definition 4.1 (Rekeying cipher map)**.** Let s_1, s_2 be distinct secrets and let $\mathrm{enc}_1, \mathrm{enc}_2$ and $\mathrm{dec}_1, \mathrm{dec}_2$ be the corresponding encoder/decoder pairs. The *rekeying cipher map* for the latent identity function $\mathrm{id}_X : X \to X$ is the cipher map

$$\hat{r} : \{0,1\}^n \to \{0,1\}^n$$

constructed as follows. For each $x \in X$ and each representation index $k \in \{0, \ldots, K(x)-1\}$, set

$$\hat{r}\big(\mathrm{enc}_1(x,k)\big) \;=\; \mathrm{enc}_2(x,k'),$$

for some choice of representation index k' in $\mathsf{C}_{s_2}(X)$. Cipher values not in $\mathsf{C}_{s_1}(X)$ (noise inputs) map to noise outputs in $\mathsf{C}_{s_2}(X)$, per the totality property.

Proposition 4.1 (Soundness of rekeying cipher map)**.** *The rekeying cipher map $\hat{r}$ from Definition 4.1 realizes the rekeying functor $r_{s_1 \to s_2}$: for every $c \in \mathsf{C}_{s_1}(X)$,*

$$\mathrm{dec}_1(c) \;=\; \mathrm{dec}_2(\hat{r}(c)).$$

Proof. Let $c \in \mathsf{C}_{s_1}(X)$. Since c is in the cipher space, $c = \mathrm{enc}_1(x,k)$ for some $x \in X$ and some representation index k, and $\mathrm{dec}_1(c) = x$. By construction, $\hat{r}(c) = \mathrm{enc}_2(x,k')$ for some k', so $\mathrm{dec}_2(\hat{r}(c)) = x$. Equality follows. □

The construction uses the trusted machine's knowledge of both secrets: it enumerates $\mathsf{C}_{s_1}(X)$, decodes each cipher value to its latent, and re-encodes under s_2. Once constructed, $\hat{r}$ is just a cipher map of the form $\{0,1\}^n \to \{0,1\}^n$, and the untrusted machine evaluates it like any other cipher map.

4.2 Construction and per-rekeying cost

Proposition 4.2 (Cost of rekeying cipher map)**.** *The rekeying cipher map $\hat{r}$ has space $O(|X|)$ entries, the same as a generic cipher map for a latent function over X [12, Sec. 6]. Per-rekeying cost is one cipher map evaluation, $O(1)$ in the cipher map's evaluation time.*

Proof. The cipher map $\hat{r}$ has one entry per latent value $x \in X$ (possibly multiplied by representation count $K(x)$ if multiple input representations need distinct outputs), matching the size of any cipher map for a function with domain X. Per-rekeying cost is one lookup in $\hat{r}$, which is the same as any cipher map evaluation. □

Remark 4.1 (Comparison with trusted-machine rekeying). Trusted-machine rekeying (Regime 1) has $O(1)$ space (no stored cipher map) and $O(\mathrm{dec} + \mathrm{enc})$ time per rekeying. Cipher-map-mediated rekeying (Regime 2) has $O(|X|)$ space and $O(1)$ time per rekeying. The trade-off is the standard cipher-map vs. on-the-fly trade-off [12, Sec. 3.3]: the lookup table costs space but saves time per evaluation.

5 Naturality: Rekeying Commutes with Operations

The categorical view of rekeying suggests it should commute with cipher operations. We make this precise.

5.1 The naturality square

Suppose $f : X \to Y$ is a latent function, with cipher maps $\mathsf{C}_{s_1}(f) : \mathsf{C}_{s_1}(X) \to \mathsf{C}_{s_1}(Y)$ and $\mathsf{C}_{s_2}(f) : \mathsf{C}_{s_2}(X) \to \mathsf{C}_{s_2}(Y)$ realizing f under each secret. Then we can either rekey first and apply f, or apply f first and rekey. Naturality says these give the same result.

Theorem 5.1 (Naturality of rekeying)**.** *For any latent function $f : X \to Y$ with cipher map realizations $\mathsf{C}_{s_1}(f), \mathsf{C}_{s_2}(f)$ under secrets s_1, s_2, the diagram*

$$\begin{array}{ccc} \mathsf{C}_{s_1}(X) & \xrightarrow{\mathsf{C}_{s_1}(f)} & \mathsf{C}_{s_1}(Y) \\ \downarrow r^X_{s_1 \to s_2} & & \downarrow r^Y_{s_1 \to s_2} \\ \mathsf{C}_{s_2}(X) & \xrightarrow{\mathsf{C}_{s_2}(f)} & \mathsf{C}_{s_2}(Y) \end{array}$$

commutes up to representation equivalence: writing $c \sim c'$ when $\mathrm{dec}_2(c) = \mathrm{dec}_2(c')$, we have, for all $c \in \mathsf{C}_{s_1}(X)$,

$$\mathsf{C}_{s_2}(f)\big(r^X_{s_1 \to s_2}(c)\big) \;\sim\; r^Y_{s_1 \to s_2}\big(\mathsf{C}_{s_1}(f)(c)\big).$$

Under perfect correctness ($\eta = 0$) both sides decode to $f(x)$ with certainty; for $\eta > 0$ they agree with probability at least $1 - \eta_{\text{total}}$, the accumulated correctness of $\mathsf{C}_{s_1}(f), \mathsf{C}_{s_2}(f)$ along the two paths. Here $r^X_{s_1 \to s_2}$ and $r^Y_{s_1 \to s_2}$ denote the rekeying functors for the type X and the type Y respectively.

Proof. Let $c \in \mathsf{C}_{s_1}(X)$ with $\mathrm{dec}_1(c) = x$. We compute both compositions.

Top-right path: apply $\mathsf{C}_{s_1}(f)$ first, then rekey.

$$\mathrm{dec}_1(\mathsf{C}_{s_1}(f)(c)) = f(x) \quad \text{(by correctness of } \mathsf{C}_{s_1}(f)\text{)}.$$

Applying $r^Y_{s_1 \to s_2}$ to $\mathsf{C}_{s_1}(f)(c)$ gives a cipher value $c' \in \mathsf{C}_{s_2}(Y)$ with $\mathrm{dec}_2(c') = f(x)$ (by soundness of the rekeying functor, Proposition 4.1).

Bottom-left path: rekey first, then apply $\mathsf{C}_{s_2}(f)$. $r^X_{s_1 \to s_2}(c) \in \mathsf{C}_{s_2}(X)$ with $\mathrm{dec}_2(\cdot) = x$. Applying $\mathsf{C}_{s_2}(f)$ gives a cipher value $c'' \in \mathsf{C}_{s_2}(Y)$ with $\mathrm{dec}_2(c'') = f(x)$ (by correctness of $\mathsf{C}_{s_2}(f)$).

Both c' and c'' are cipher values in $\mathsf{C}_{s_2}(Y)$ that decode to $f(x)$, hence $c' \sim c''$. This is the standard sense in which cipher map diagrams commute: up to the equivalence relation that identifies cipher values with the same latent value, the two paths agree. □

Corollary 5.2 (Operational consequence)**.** *A computation pipeline $\hat{f}_n \circ \cdots \circ \hat{f}_1$ on cipher values in $\mathsf{C}_{s_1}(X)$ can be rekeyed to s_2 at any point in the pipeline (including before, between any two operations, or after the last operation), and the final cipher value's decoded latent value is the same.*

Proof. Iterated application of Theorem 5.1: insert $r_{s_1 \to s_2}$ at any point and use naturality to move it to the beginning (or end) of the pipeline. □

5.2 Rekeying chains

A sequence of rekeyings $s_0 \to s_1 \to s_2 \to \ldots \to s_n$ is a *rekeying chain*. Naturality and composition give a clean compositional account.

Proposition 5.3 (Composition of rekeying functors)**.** *For any three secrets s_1, s_2, s_3,*

$$r_{s_1 \to s_3} \;=\; r_{s_2 \to s_3} \circ r_{s_1 \to s_2},$$

in the sense that decoding under s_3 after applying both rekeyings agrees with decoding under s_3 after applying the direct $r_{s_1 \to s_3}$.

Proof. Let $c \in \mathsf{C}_{s_1}(X)$ with $\mathrm{dec}_1(c) = x$. By soundness of each rekeying: $\mathrm{dec}_2(r_{s_1 \to s_2}(c)) = x$, $\mathrm{dec}_3(r_{s_2 \to s_3}(r_{s_1 \to s_2}(c))) = x$. Direct application: $\mathrm{dec}_3(r_{s_1 \to s_3}(c)) = x$. Both yield cipher values in $\mathsf{C}_{s_3}(X)$ with decoded value x, so they agree up to representation choice. □

6 Information-Theoretic Cost

Rekeying is not free. Each application of $\hat{r}$ produces a cipher value in a new cipher space that decodes to the same latent value as the input. The pair $(c, \hat{r}(c))$ is observable to any party that sees both cipher values, and that pair carries information. We characterize this leakage and show how to mitigate it.

6.1 Cross-cipher-space correlation

Proposition 6.1 (Rekeying as orbit expansion)**.** *Let F be a set of cipher maps including $\hat{r}$. Then for any cipher value $c \in \mathsf{C}_{s_1}(X)$, the orbit* $\mathrm{orbit}_F(c)$ *includes both c and $\hat{r}(c)$, plus any further images under operations in F. The orbit-closure bound [11, Thm. 5.3] applies to the expanded F.*

Proof. Direct: by the orbit-closure definition of [11], the orbit under F is the smallest set containing c that is closed under all operations in F. Including $\hat{r}$ in F adds $\hat{r}(c)$ to the orbit. Since $\hat{r}(c) \in \mathsf{C}_{s_2}(X)$ rather than $\mathsf{C}_{s_1}(X)$, subsequent operations from F that land in $\mathsf{C}_{s_2}(X)$ further expand the orbit; and so on. □

Remark 6.1 (The leakage shape)*.* What rekeying specifically leaks is the *cross-cipher-space correlation*: an adversary observing many pairs $(c_i, \hat{r}(c_i))$ can build the joint distribution over $(\mathsf{C}_{s_1}(X), \mathsf{C}_{s_2}(X))$ pairs. This distribution converges, in the limit of many samples, to a mixture of the joint $(\mathrm{enc}_1(x, \cdot), \mathrm{enc}_2(x, \cdot))$ for $x \sim D$. The adversary does not learn x directly, but they do learn the *paired structure* of the two cipher spaces. This is the cross-space analogue of the shared-variable correlation leakage in [11, Rem. 6.1].

6.2 Quantification: rekeying chain leakage

Theorem 6.2 (Rekeying chain confidentiality bound)**.** *Let $c_0 \in \mathsf{C}_{s_0}(X)$ encode latent value $x \in X$, and let $c_i = \hat{r}_i(c_{i-1}) \in \mathsf{C}_{s_i}(X)$ for $i = 1, \ldots, n$ be the result of an n-step rekeying chain. Let $F = \{\hat{r}_1, \ldots, \hat{r}_n\}$ be the set of rekeying maps, and let $\mathcal{V}_F = \{c_0, c_1, \ldots, c_n\}$ be the adversary's view. Then the residual entropy of X given $\mathcal{V}_F$ satisfies*

$$H(X \mid \mathcal{V}_F) \;\geq\; H(X) - \log_2(n+1).$$

Proof. The chain is a typed composition chain in the sense of [11, Sec. 5.5]: each $\hat{r}_i$ maps between the distinct cipher spaces $\mathsf{C}_{s_{i-1}}(X)$ and $\mathsf{C}_{s_i}(X)$, so the only cipher values reachable from the single starting value c_0 are $c_0, c_1, \ldots, c_n$. By the single-value-start case of [11, Prop. 5.4] (one initial cipher value, $m = 1$, depth $k = n$), the orbit therefore satisfies $|\mathrm{orbit}_F(c_0)| \leq n + 1$. Applying the entropy-form confidentiality bound of [11, Thm. 5.3] gives $H(X \mid \mathcal{V}_F) \geq H(X) - \log_2 |\mathrm{orbit}_F(c_0)| \geq H(X) - \log_2(n+1)$. □

The inequality is the orbit-closure specialization of the standard equivalence-class bound from quantitative information flow [10]: resolving the latent value only up to a reachable set of size $n+1$ leaves at least $H(X) - \log_2(n+1)$ bits of uncertainty. The novel content is not the inequality but the identification of the $(n+1)$-element orbit produced by an n-step rekeying chain.

The bound is loose for small n but tight as $n \to \infty$ when the cipher maps in the chain are independent. For practical key-rotation policies (e.g., one rekeying per day for a year, $n = 365$), the bound gives $H(X \mid \mathcal{V}_F) \geq H(X) - \log_2 366 \approx H(X) - 8.5$ bits. For a latent type with $H(X) > 8.5$ bits the bound is meaningful; for smaller types more care is needed.

6.3 Sub-Turing computation as a security feature

Theorem 6.2 ties the confidentiality guarantee to a complexity restriction. The chain length n is bounded by the number of rekeying maps the trusted machine has provisioned, and the trusted machine controls whether to provision more; programs whose composition depth grows with input size therefore cannot run without its cooperation. The framework as developed here is thus *not Turing-complete*: it offers bounded-depth straight-line composition over the cipher spaces $\mathsf{C}_{s_0}(X), \mathsf{C}_{s_1}(X), \ldots$, with depth n fixed at provisioning time. In return, each unit of depth carries an explicit, hardness-free information-theoretic price of $\log_2(n+1)$ bits per chain.

We read this as a feature, the same stance cipher maps takes toward bounded composition depth within a single secret [12, Sec. 9.4], here lifted to chains *across* secrets. Sub-Turing models recur in security and verification (capability machines, total functional languages, structurally recursive type theories, primitive-recursive complexity classes), each trading expressivity for a static guarantee; cipher rekeying makes the analogous trade information-theoretically, with n the expressivity dial and $\log_2(n+1)$ the leakage cost. The envelope matches the workloads cipher maps target (encrypted search, fixed-depth classification cascades, bounded-depth encrypted inference, pre-scheduled key rotation); workloads needing unbounded computation fall outside it, requiring fully homomorphic encryption [6] or a computational security argument instead.

Connection to cipher maps composition. Within a single cipher space, the cipher maps framework [12] permits unbounded composition: any chain $\hat{f}_1 \circ \cdots \circ \hat{f}_m$ is again a cipher map ([12, Thm. 7.2]). But same-secret composition accumulates correlation leakage that Theorem 6.2 does not capture: shared inputs reveal correlation structure even when each map has uniform marginal output ([12, §8]). Rekeying breaks this accumulation at the cost of $\log_2(n+1)$ bits per step. The two are complementary: same-secret composition is bounded by error accumulation, cross-secret rekeying by the chain bound, and together they bound what an adversary observing a multi-secret program learns.

6.4 Mitigation: randomized rekeying

The bound in Theorem 6.2 treats each rekeying as a deterministic operation that adds exactly one cipher value to the adversary's orbit. Multiple representations smear this.

Definition 6.1 (Randomized rekeying)**.** A *randomized rekeying functor* $\tilde{r}_{s_1 \to s_2}$ is a function from $\mathsf{C}_{s_1}(X)$ to $\mathsf{C}_{s_2}(X)$ that, on input $c = \mathrm{enc}_1(x, k)$, outputs $\mathrm{enc}_2(x, k')$ where k' is drawn uniformly at random from the representation indices of x in $\mathsf{C}_{s_2}(X)$.

Proposition 6.3 (Randomized rekeying breaks representation linkage)**.** *Let $c = \mathrm{enc}_1(x, k)$ and let $\tilde{r}_{s_1 \to s_2}$ be a randomized rekeying (Definition 6.1) with target representation count $K_2(x)$. Then the output representation index k' is independent of the input index k given the latent value x: $I(k; k' \mid x) = 0$, and the conditional distribution of $\tilde{r}_{s_1 \to s_2}(c)$ given c is uniform on the $K_2(x)$ representations of x in $\mathsf{C}_{s_2}(X)$.*

Proof. By construction k' is drawn uniformly from the representation indices of x in $\mathsf{C}_{s_2}(X)$, independently of k, so $k' \perp k \mid x$ and $I(k; k' \mid x) = 0$. The conditional distribution of $\tilde{r}_{s_1 \to s_2}(c)$ given c

is then uniform on those $K_2(x)$ representations — the maximum-entropy choice, in the sense of the representation-uniformity property [12, Sec. 4.2] (the randomized-encoding defense of [12, Sec. 8.3] is the same mechanism applied to rekeying). □

Remark 6.2 (Unlinkability is a different lever from the chain bound). Proposition 6.3 reduces *cross-space linkability*: an adversary cannot match an input representation to its output representation beyond what the latent value already forces. This is a different lever from the orbit-size bound of Theorem 6.2. Fanning each latent value out over $K_2(x)$ target representations can in fact *enlarge* the orbit (more distinct cipher values become reachable), so randomized rekeying buys unlinkability without tightening — and potentially loosening — the chain bound itself. The two are complementary: Theorem 6.2 governs how tightly the *number* of reachable cipher values pins down the latent value, while Proposition 6.3 governs how much the *pairing* between successive cipher spaces leaks.

The cost of randomized rekeying is the number of representations $K_2(x)$ in $\mathsf{C}_{s_2}(X)$. The space accounting is the multiple-representations trade-off of [14]: more representations cost more space; here they buy cross-space unlinkability (Proposition 6.3) rather than marginal uniformity.

6.5 When is rekeying-induced leakage acceptable?

The leakage bound is an upper bound; for many applications the practical cost is much lower. Three regimes deserve mention.

- **Few rekeyings, large latent type.** When n rekeyings are performed and $H(X) \gg \log_2(n+1)$, the bound is comfortably satisfied. Standard key rotation (a few rekeyings per cipher value's lifetime) on rich latent types (file contents, database records) falls here.
- **Many rekeyings, randomized.** With randomized rekeying and many representations per latent value, the per-rekeying leakage drops below 1 bit, and chains of length $\Omega(2^{H(X)})$ are required before the bound becomes a practical concern.
- **Adversarial rekeying.** An adversary that can request rekeyings on chosen cipher values (chosen-input attack) can probe the rekeying functor in ways that passive observation cannot. This is outside the scope of the orbit-closure analysis and deserves separate treatment.

7 Applications

We sketch four applications that motivate the rekeying primitive.

7.1 Scheduled key rotation

A long-running system rotates its secret on a schedule. At rotation time the trusted machine generates s_{new}, constructs the rekeying cipher map $\hat{r} : \mathsf{C}_{s_{\text{old}}}(X) \to \mathsf{C}_{s_{\text{new}}}(X)$, and the untrusted machine applies $\hat{r}$ to all stored cipher values; the trusted machine then retires s_{old} (deleting $\text{enc}_{\text{old}}, \text{dec}_{\text{old}}$ and the secret). The leakage cost is one orbit step per rotation (Theorem 6.2).

7.2 Multi-tenant cipher systems

Two parties with their own secrets wish to share cipher values without sharing secrets. A trusted intermediary holding both constructs a rekeying cipher map $\hat{r} : \mathsf{C}_{s_A}(X) \to \mathsf{C}_{s_B}(X)$; thereafter the rekeying runs on untrusted infrastructure, and neither party learns the other's secret.

The benefit is *secret isolation and operational independence*, not plaintext-level confidentiality between A and B: party B holds s_B and can decode any value in $\mathsf{C}_{s_B}(X)$, rekeyed ones included. What the construction buys is that A and B each operate within their own cipher machinery without exchanging secrets, with the intermediary's role limited to the one-time construction of $\hat{r}$. The leakage cost per value transferred is one orbit step (Proposition 6.1).

7.3 Forward secrecy via key retirement

Retiring an old secret s_{old} provides forward secrecy for *archived* cipher values: values that existed only as $\mathsf{C}_{s_{\text{old}}}(X)$ encodings (offline backups or external archives the rotation did not touch) cannot be decoded by an adversary who compromises s_{new}, provided s_{old} and the intermediate rekeying maps $\hat{r}_{\text{old}\to\text{new}}$ were deleted at rotation time. Those maps are themselves cipher maps; deleting them severs the path from $\mathsf{C}_{s_{\text{old}}}(X)$ to $\mathsf{C}_{s_{\text{new}}}(X)$, so an adversary holding only s_{new} cannot bring archived values into decodable form.

This does *not* cover values that were rekeyed and remain in $\mathsf{C}_{s_{\text{new}}}(X)$: those are exactly what s_{new} decodes, and its compromise exposes them. Forward secrecy here is a property of the *retirement*, not of rekeying: once a cipher space is retired (secret and live-space rekeying maps deleted), the values it held become unreachable from any later cipher machinery.

7.4 Rekeying chains for evolving trust boundaries

A cipher value may pass through several trust domains over its lifetime, each with its own secret. The rekeying-chain composition (Proposition 5.3) lets the value's lineage be tracked as a sequence $s_0 \to s_1 \to \cdots \to s_n$ of secrets, with the cipher value at each step in the corresponding cipher space. The total leakage is bounded by Theorem 6.2. This supports use cases like data shared across organizational boundaries with hand-offs at each boundary.

8 Related Work

Proxy re-encryption. Proxy re-encryption (PRE), originating with Blaze, Bleumer, and Strauss [3], addresses the same shape of problem as cipher rekeying: convert a ciphertext under one key to a ciphertext under another key without decrypting. Modern PRE constructions (Ateniese et al. [2] and successors) achieve this with computational hardness assumptions and a re-encryption key that's distributable to a third-party proxy. Cipher rekeying, by contrast, makes information-theoretic guarantees and requires the trusted machine to know both secrets at construction time. The two approaches sit at different points in the design space:

- PRE is asymmetric and computationally secure; rekeying is symmetric and information-theoretic.
- PRE supports public-key proxy operation; rekeying assumes the rekeying map is constructed by a trusted party.
- PRE composes within an algebraic key structure; rekeying composes by orbit-closure analysis.

For applications where computational hardness is acceptable and public-key proxy operation is desired, PRE is the right primitive. For applications already using the trapdoor-computing framework, where information-theoretic guarantees are required and a trusted party is available, rekeying is more direct.

Functional encryption. Functional encryption [4] allows computing $f(x)$ on an encrypted x using a function-specific key. A function-specific decryption key is conceptually similar to a rekeying map specialized to a particular function: both are tools for changing the relationship between a ciphertext and its decoded output. The information-theoretic vs. computational distinction applies here as well: FE is computational, cipher operations in [12] and rekeying here are information-theoretic.

Key-homomorphic PRFs. Key-homomorphic PRFs [5] allow combining keys algebraically: $F(k_1 + k_2, x) = F(k_1, x) + F(k_2, x)$ for a suitable group operation $+$. This supports a form of distributed key management where multiple parties contribute key shares. The rekeying functor in this paper is structurally similar to a key-rotation operation that could be implemented with a key-homomorphic PRF, though the trapdoor-computing setting offers a different security guarantee (information-theoretic, with a representation-uniformity property) than KH-PRF (computational, with pseudorandomness).

Information flow types. The closures-as-data-structures framing of cipher data echoes the type-systems-for-information-flow approach of Sabelfeld and Myers [9]: types annotate values with security levels, and a type system enforces non-interference. Cipher closures take this further by parameterizing types by a secret: each cipher closure type carries its secret as a parameter, and rekeying changes the parameter. This is a structural type-level operation that information-flow type systems do not directly express.

ORAM and access patterns. Oblivious RAM [7] hides access patterns between an untrusted storage server and a trusted client. Rekeying is orthogonal: it changes the secret a cipher value is encoded under, not the access pattern to the cipher value. A system can use both: ORAM to hide which cipher values are accessed, and rekeying to rotate the secret used to encode them.

9 Conclusion

We have developed cipher rekeying: a transformation between cipher values encoded under different secrets, framed through the closures-as-data-structures lens of SICP. The construction is straightforward when the trusted machine knows both secrets: the rekeying functor is itself a cipher map, with $O(|X|)$ space and $O(1)$ time per rekeying. The functor commutes with arbitrary cipher operations (naturality), composes cleanly into chains, and admits a precise information-theoretic accounting via the orbit-closure framework of [11].

The closure framing is more than rhetorical. It exhibits rekeying as one instance of treating the secret as captured state, the secret-as-data corner of a broader code–data duality for cipher closures (cipher data structures as cipher maps, and cipher maps as cipher data) developed separately [13]. For the trapdoor-computing program more broadly, treating the secret as data (rather than parameter) opens a class of operations, including but not limited to rekeying, that are missing from the current literature. A natural sequel is to push this further: index cipher operations by secret, allow multiple secrets to coexist within a single closure, and analyze leakage with respect to secret-equality queries.

The primary practical contribution is a rekeying primitive with information-theoretic guarantees that fits the trapdoor-computing framework directly. Applications include scheduled key rotation, multi-tenant cipher systems, forward secrecy via key retirement, and rekeying chains for evolving trust boundaries.

Acknowledgments

The cipher-rekeying construction grew out of the author's earlier unpublished work on algebraic cipher types; this paper develops and formalizes it in the language of the cipher-map framework [12] and the algebraic cipher types extension [11].

References

[1] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. MIT Press, 2 edition, 1996.

[2] Giuseppe Ateniese, Kevin Fu, Matthew Green, and Susan Hohenberger. Improved proxy re-encryption schemes with applications to secure distributed storage. volume 9, pages 1–30, 2006.

[3] Matt Blaze, Gerrit Bleumer, and Martin Strauss. Divertible protocols and atomic proxy cryptography. In *International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, pages 127–144, 1998.

[4] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *Theory of Cryptography Conference (TCC)*, pages 253–273, 2011.

[5] Dan Boneh, Kevin Lewi, Hart Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In *Annual International Cryptology Conference (CRYPTO)*, pages 410–428, 2013.

[6] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.

[7] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

[8] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 2 edition, 1978.

[9] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[10] Geoffrey Smith. On the foundations of quantitative information flow. In *Proceedings of the 12th International Conference on Foundations of Software Science and Computational Structures (FoSSaCS)*, pages 288–302, 2009.

[11] Alexander Towell. Algebraic cipher types: Confidentiality trade-offs in type constructors over trapdoor computing. Zenodo, 2026. URL `https://doi.org/10.5281/zenodo.20607285`.

[12] Alexander Towell. Cipher maps: Total functions as trapdoor approximations, 2026. Manuscript in preparation.

[13] Alexander Towell. Cipher closures: Cipher data structures and the code-data duality in trapdoor computing, 2026. Manuscript in preparation.

[14] Alexander Towell. Maximizing confidentiality under trapdoor computing: Encoding granularity and optimal space. Zenodo, 2026. URL `https://doi.org/10.5281/zenodo.20607278`.